R AND PYTHON

ID:12310007

REG:000018839

Name:Moreom Akter Mohona

R Code:

Question-1:What is the output of the following code?

Ans:

R Code:
```
x <- c(1,2,3,4,5)
cat(length(x), class(x))
```

R Output: 5 numeric

Correct Answer: A. 5 numeric

Question-2:Which assignment operators are valid in R?

Ans:

R Code:
```
x <- 5
y = 10
15 -> z
print(x)
print(y)
print(z)
```

R Output:
```
[1] 5
[1] 10
[1] 15
```

Correct Answer: D. <-, =, and -> are all valid

Question-3:Difference between NA, NULL and NaN in R?

Ans:

R Code:
```
x <- NA
y <- NULL
z <- NaN
print(x)
print(y)
print(z)
```

R Output:
```
NA
NULL
NaN
```

Correct Answer: B. NA = missing, NULL = empty, NaN = undefined number

Question-7:What does the following code return?

Ans:

R Code: x <- c(10,20,30,40,50)
x[c(2,4)]

R Output: 20 40

Correct Answer:A. 20 40

Question- 8: What is the result of this code?

Ans:

R Code: x <- 1:5

x > 3

R Output: FALSE FALSE FALSE TRUE TRUE

Correct Answer: C. FALSE FALSE FALSE TRUE TRUE

Question-9:What does Negative indexing in R?

Ans:

R Code: x <- c(5,10,15,20)

x[-2]

R Output: 5 15 20

Correct Answer: B. Returns 5 15 20

Question-13:What does this code produce?

Ans:

R Code: df <- data.frame(x=1:3, y=c("a","b","c"))

nrow(df)

ncol(df)

R Output: 3 2

Correct Answer: A. 3 2

Question-16:What is the output of this function call?

Ans:

R Code: add <- function(a, b=10){

(a+b) return} add(5)

R Output: 15

Correct Answer: B. 15

Question-24:Which function performs a two-sample t-test in R?

Ans:

R Code: A group <- c(10,12,14
groupB <- c(11,13,15)
t.test(groupA, groupB)
R Output: 12 13

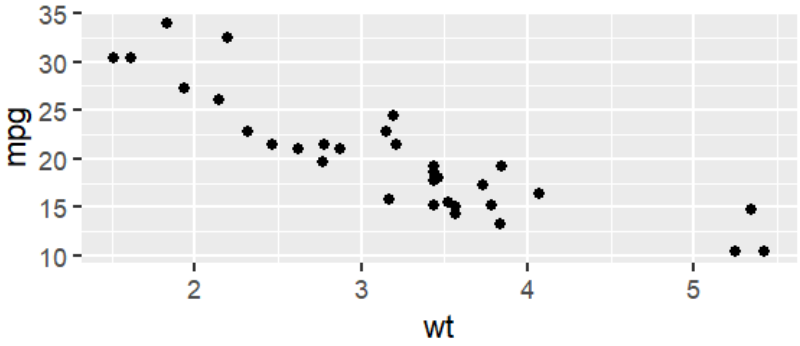
Correct Answer: A. t.test()

Question-29:Which ggplot2 function adds a scatter plot layer?

Ans:

R Code: library(ggplot2)
ggplot(mtcars, aes(x=wt, y=mpg)) +
geom_point()

R Output:



Correct Answer: A. geom_point()

Question-32: What is the output of the following code?

Ans:

R Code: nchar("Hello, R!")

R Output: `[1] 9`

Correct Answer: 9.

Question-33:What is the output of this code?

Ans:

R Code: x <- "Data Science"
toupper(substring(x, 1, 4))
R Output: [1] "DATA"

Correct Answer: A. "DATA"

Question-34:Which function replaces ALL occurrences of a pattern in a string?

Ans:

R Code: x <- "the cat sat on the mat"
gsub("at", "og", x)
R Output: [1] "the cog sog on the mog"

Correct Answer: B. gsub()

Question-35:What does this stringr code produce?

Ans:

R Code: library(stringr)
str_split("a,b,c,d", ",")
R Output: [[1]][1] "a" "b" "c" "d"

Correct Answer: B. A list containing a character vector: c("a","b","c","d")

Question-36:How do you detect if a pattern exists in a string in base R?

Ans:

R Code: x <- c("apple", "banana", "cherry")
grepl("an", x)
R Output: [1] FALSE TRUE FALSE

Correct Answer: C. grepl('an', x)

Question-37:What does paste() vs paste0() do differently?

Ans:

R Code: paste("R","is","great")
paste0("R","is","great")
R Output: [1] "R is great"[1] "Risgreat"

Correct Answer: B. paste() joins with a space by default; paste0() joins with no separator.

Question-38:What is the output of this for loop?

Ans:

R Code:
```
total <- 0
for(i in 1:5){
  total <- total + i
}
cat(total)
```
R Output: 15

Correct Answer: C. 15

Question-39:What does break do inside a loop?

Ans:

R Code:
```
for(i in 1:10){
  if(i == 4) break
  cat(i," ")
}
```
R Output: 1 2 3

Correct Answer: B. Prints 1 2 3

Question-40:What is the output of this while loop?

Ans:

R Code:
```
x <- 1
while(x < 5){
  x <- x * 2
}
cat(x)
```
R Output: 8

Correct Answer: B. 8

Question-42:What does apply() do with MARGIN = 1 vs MARGIN = 2?

Ans:

R Code:
```
m <- matrix(1:9, nrow=3)
apply(m,1,sum)
apply(m,2,sum)
```
R Output: [1] 12 15 18
[1] 6 15 24

Correct Answer: B. MARGIN=1 applies across rows; MARGIN=2 applies across columns

Question-43:What does tapply() do?

Ans:

R Code:
```
scores <- c(85,90,78,92,88)
group <- c("A","B","A","B","A")
tapply(scores, group, mean)
```
R Output:
```
       A        B
83.66667 91.00000
```
Correct Answer: B. Applies a function to subsets of a vector defined by a grouping factor.

Question-44:What is the output of this Map() call?

Ans:

R Code: result <- Map(function(x,y) x+y,
c(1,2,3),
c(10,20,30))
result

R Output: [[1]]
[1] 11
[[2]]
[1] 22
[[3]]
[1] 33

Correct Answer: B. A list: list(11,22,33)

Question-45:What is 'recycling' in R?

Ans:

R Code: c(1,2,3,4) + c(10,20)

R Output:[1] 11 22 13 24

Correct Answer: B. When R automatically repeats shorter vectors to match the length of a l onger one.

Question-46:When should you use a chi-squared test in R?

Ans:

R Code: chisq.test(table(gender, preference))

R Output: Chi-squared test
p-value = 0.032

Correct Answer: B. To test association between two categorical variables.

Question-48:What is the non-parametric alternative to a two-sample t-test?

Ans:

R Code: wilcox.test(groupA, groupB)

R Output: Wilcoxon rank sum test
p-value = 0.041

Correct Answer: B. Mann-Whitney U test (Wilcoxon rank-sum test)

Question-52:Which function fits a logistic regression model in R?

Ans:

R Code:model <- glm(outcome ~ age + gender,
data=df,
family=binomial)

R Output:Model fitted successfully

Correct Answer: B. glm()

Question-53:What does a confidence interval represent?

Ans:

R Code:t.test(x)$conf.int

R Output: [1] 3.21 7.45
attr(,"conf.level")
[1] 0.95
Correct Answer: B. A range of values that would contain the true population parameter in 9 5% of repeated samples.
Question-54:What is a paired t-test used for?

Ans:

R Code:t.test(before, after, paired = TRUE)

R Output: Paired t-test
p-value = 0.018
Correct Answer: B. Comparing means of the same subjects measured twice.

Python Code:

Q1. What is the output of the following code?

x = 5

y = 2

print(x // y, x % y)

Solution:

Python Code:x = 5

y = 2

print(x // y, x % y)

Output: 2 1

Q2: Write a Python expression to reverse the string s = "Python".

Solution:

Python Code: s = "Python"

reversed_s = s[::-1]

print(reversed_s)

Output: nohtyp

Q3. What is the difference between a list and a tuple in Python? Give one example of each.

Solution:

Python Code :# List — mutable

```
my_list = [1, 2, 3]
my_list[0] = 99 # allowed
# Tuple — immutable
my_tuple = (1, 2, 3)
# my_tuple[0] = 99 # raises TypeError
```

Q4. Write a Python program that checks if a given number is even or odd.

Solution:

Python Code:

```
num = int(input("Enter a number: "))
if num % 2 == 0:
 print(num, "is even")
else:
 print(num, "is odd")
Result: Enter a number:  5
5 is odd
```

Q5. Write a loop that prints the numbers 1 to 10, but skips multiples of 3.

Solution:

Python Code:

```
for i in range(1, 11):
 if i % 3 == 0:
 continue
```

print(i)

Output: 1 2 4 5 7 8 10

Q6: Write a Python function factorial(n) that returns the factorial of n using recursion.

Solution:

Python Code:
```
def factorial(n):
    if n == 0 or n == 1:
    return 1
    return n * factorial(n - 1)
print(factorial(5)) # 120
print(factorial(0)) # 1
```

Result: 120 1

Q7. Given a list nums = [3, 1, 4, 1, 5, 9,2,6]write a one -liner to get a sorted list of unique values.

Solution:

Python Code:
```
nums = [3, 1, 4, 1, 5, 9, 2, 6]
result = sorted(set(nums))
print(result)
```

Output: [1, 2, 3, 4, 5, 6, 9]

Q8. Write a function that checks whether a string is a palindrome (reads the same forwards and backwards), ignoring case.

Solution:

Python Code:
```
def is_palindrome(s):
    s = s.lower()
    return s == s[::-1]
print(is_palindrome("Racecar")) # True
```

print(is_palindrome("Python")) # False

Result: True False

Q9: Use a list comprehension to create a list of squares of all even numbers from 0 to 20 (inclusive).

Solution:

Python Code: squares = [x**2 for x in range(0, 21) if x % 2 == 0]

print(squares)

Output: [0, 4, 16, 36, 64, 100, 144, 196, 256, 324, 400]

Q10. What is the output of the following code? Explain why.

def foo(x, lst=[]):

lst.append(x)

return lst

print(foo(1))

print(foo(2))

print(foo(3))

Solution:

Python Code: def foo(x, lst=[]):

lst.append(x)

return lst

print(foo(1)) # [1]

print(foo(2)) # [1, 2]

print(foo(3)) # [1, 2, 3]

Result: [1] [1,2] [1,2,3]

Q11. Write a function that counts how many times each word appears in a sentence and returns the result as a dictionary.

Solution:

Python Code:

```
def word_count(sentence):
    words = sentence.lower().split()
    counts = {}
    for word in words:
    counts[word] = counts.get(word, 0) + 1
    return counts
print(word_count("the cat sat on the mat the cat"))
# {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat':1}
```

Result: {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1}

Q12. Define a class Animal with a name attribute and a speak() method. Then create a subclass Dog that overrides speak() to return 'Woof!'.

Solution:

Python Code:

```
class Animal:
    def __init__(self, name):
    self.name = name
    def speak(self):
    return f"{self.name} makes a sound."
class Dog(Animal):
    def speak(self):
    return f"{self.name} says: Woof!"
a = Animal("Generic")
d = Dog("Rex")
print(a.speak()) # Generic makes a sound.
print(d.speak()) # Rex says: Woof!
```

Result: Generic makes a sound.

Rex says: Woof!

Q13: Write a decorator @timer that prints the time taken to execute any function it decorates.

Solution:

Python Code:

```
import time

def timer(func):
 def wrapper(*args, **kwargs):
 start = time.time()
 result = func(*args, **kwargs)
 end = time.time()
print(f"{func.__name__} took {end - start:.4f}s")
 return result
 return wrapper
@timer
def slow_sum(n):
 return sum(range(n))
slow_sum(1_000_000)
# slow_sum took 0.0312s (time will vary)
```

Result:

```
slow_sum took 0.0482s
499999500000
```

Q14. Write a function flatten(lst) that takes a nested list of arbitrary depth and returns a flat list.

Solution:

Python Code:

```
def flatten(lst):
 result = []
 for item in lst:
```

```
 if isinstance(item, list):
 result.extend(flatten(item))
 else:
 result.append(item)
 return result
nested = [1, [2, [3, [4]], 5], 6]
print(flatten(nested))
```

Output: [1, 2, 3, 4, 5, 6]

Q15. Implement a Stack class using a Python list, with push(), pop(), peek(), and is_empty() methods. Make it iterable.

Solution:

Python Code:

```
class Stack:
    def __init__(self):
    self._data = []
    def push(self, item):
    self._data.append(item)
   def pop(self):
    if self.is_empty():
    raise IndexError("pop from empty stack")
    return self._data.pop()
    def peek(self):
    if self.is_empty():
    raise IndexError("peek at empty stack")
    return self._data[-1]
    def is_empty(self):
    return len(self._data) == 0
    def __iter__(self):
```

```
 return reversed(self._data)
 def __len__(self):
 return len(self._data)
s = Stack()
s.push(1); s.push(2); s.push(3)
print(s.peek()) # 3
print(list(s)) # [3, 2, 1]
print(s.pop()) # 3
```

Result:3 [3,2,1] 3